

The Non-Analytic Critical-Region Term

Part 2: Numerical Methods

outram-park-fork-coolprop

2026-07-10

Abstract

Part 1 introduced the non-analytic term's three auxiliary functions θ , Δ , ψ and their physical role. This document works through the first- and second-derivative chain needed for pressure, c_v , c_p , and the speed of sound, and the numerical guard required to evaluate those derivatives *at* the critical point itself, where several intermediate quantities are formally singular.

1 What derivatives are actually needed

Every property this crate computes from the Helmholtz surface — pressure, internal energy, entropy, both heat capacities, the speed of sound — is built from the five quantities

$$\alpha, \quad \alpha_\delta = \frac{\partial \alpha}{\partial \delta}, \quad \alpha_\tau = \frac{\partial \alpha}{\partial \tau}, \quad \alpha_{\delta\delta} = \frac{\partial^2 \alpha}{\partial \delta^2}, \quad \alpha_{\tau\tau} = \frac{\partial^2 \alpha}{\partial \tau^2}, \quad \alpha_{\delta\tau} = \frac{\partial^2 \alpha}{\partial \delta \partial \tau}$$

(the crate's `HelmholtzDerivs` struct — see Part 3). Every residual term type, including the non-analytic one, must supply all six.

2 The derivative chain

Differentiating Eq. (3) from Part 1 term by term is mechanical but long; the key structural fact that keeps it tractable is that every derivative can be written as $\alpha_{r,i}$ times a *ratio* function, which itself has a clean recursive structure:

$$\frac{\partial \theta}{\partial \tau} = -1, \quad \frac{\partial \theta}{\partial \delta} = \frac{A_i}{\beta_i} [(\delta - 1)^2]^{\frac{1}{2\beta_i} - 1} (\delta - 1), \quad (1)$$

$$\frac{\partial \Delta^{b_i}}{\partial \delta} = b_i \Delta^{b_i-1} \frac{\partial \Delta}{\partial \delta}, \quad \frac{\partial \Delta^{b_i}}{\partial \tau} = -2 b_i \theta \Delta^{b_i-1}. \quad (2)$$

The full second-derivative expressions (implemented directly in `eos.rs`, following CoolProp's `Helmholtz.cpp`) combine these with the ordinary product rule applied to $\alpha_{r,i} = n_i \delta \Delta^{b_i} \psi$. The pattern throughout is: *every* derivative is some polynomial combination of θ , Δ , Δ^{b_i-1} , Δ^{b_i-2} , and the corresponding δ/τ derivatives of θ and ψ — nothing beyond elementary calculus, just a long chain of it.

3 The branch point: why $\delta = 1$, $\tau = 1$ is singular

Look again at $\partial\theta/\partial\delta$ above. As $\delta \rightarrow 1$, $(\delta - 1)^2 \rightarrow 0$, and the exponent $\frac{1}{2\beta_i} - 1$ is applied to that vanishing quantity. For IAPWS-95 Water ($\beta_i = 0.3$), this particular exponent is ≈ 0.67 — positive, so this specific expression is actually fine at $\delta = 1$ (it evaluates to exactly 0). The *second* δ -derivative of θ , needed for $\alpha_{\delta\delta}$, instead uses the exponent $\frac{1}{2\beta_i} - 2 \approx -0.33$ — *negative*

— so $[(\delta - 1)^2]^{-0.33}$ diverges as $\delta \rightarrow 1$: $0^{-0.33} \rightarrow \infty$. A symmetric problem appears in the Δ^{b_i-2} factor used by the second- δ -derivative of Δ^{b_i} . In exact arithmetic, these divergences are known to cancel against compensating factors elsewhere in the same expression (the overall $\alpha_{r,i}$ and its first two derivatives remain finite at the critical point) — but evaluating the raw intermediate quantities in floating point *at exactly* $\delta = 1$ hits a literal 0^{negative} (effectively infinite) before those cancellations can happen.

4 The guard: offset by a few ULPs

CoolProp’s own resolution (`Helmholtz.cpp`, ported verbatim into this crate’s `accumulate_non_analytic`) is not to try to derive the closed-form limit of every intermediate quantity — it is simpler and numerically robust to just *avoid* evaluating exactly at the branch point:

```
let delta = if (delta - 1.0).abs() < 10.0 * f64::EPSILON {
    1.0 + 10.0 * f64::EPSILON
} else {
    delta
};
let tau = if (tau - 1.0).abs() < 10.0 * f64::EPSILON {
    1.0 + 10.0 * f64::EPSILON
} else {
    tau
};
```

Ten units in the last place ($10 \times \text{f64::EPSILON} \approx 2.22 \times 10^{-15}$) is far smaller than any physically meaningful state resolution, so this offset is invisible to any real calculation — but it is large enough that IEEE 754 double-precision division and fractional exponentiation near that point behave as ordinary (very large but finite) numbers rather than literal infinities or NaNs, and the term’s genuine *finite* limiting value falls out correctly from the surrounding arithmetic.

5 Why this is trustworthy: verification strategy

A guard like this is easy to get subtly wrong (e.g. offsetting the wrong variable, or by the wrong sign), so it is verified two ways, both covered in Part 4: first, by evaluating *exactly* at Water’s tabulated critical point (T_c, ρ_c) — the offset guard fires on *both* δ and τ simultaneously, the most demanding case — and checking the resulting pressure against IAPWS-95’s defining value; second, by evaluating one degree below T_c on the critical isochore, where the guard does not fire at all, as a monotonicity sanity check that the two code paths agree in the limit.